

Article

A Lightweight Framework for Android Malware Detection via SDAE-Based Multi-View Static Feature Fusion

Man Hua, Yanhang Shi * and Yanling Li

School of Computer Science and Artificial Intelligence, Civil Aviation Flight University of China, Guanghan 618307, China; huaman@cafuc.edu.cn (M.H.); liyanling@cafuc.edu.cn (Y.L.)

* Correspondence: shiyanhang@cafuc.edu.cn

Abstract

Android malware detection is increasingly important for mobile and edge security because malicious applications may compromise user privacy, device reliability, and sensitive service transactions. However, single-view static detection methods often provide limited semantic coverage and are sensitive to noisy or obfuscated code, while many deep learning models remain too heavy for resource-constrained deployment. To address these challenges, this paper proposes a lightweight Android malware detection framework based on SDAE-guided multi-view static feature fusion. The framework extracts three complementary static views, namely API calls, permission requests, and system components, from AndroidManifest.xml and classes.dex. These views are independently denoised and compressed by stacked denoising autoencoders, then aligned as the R, G, and B channels of a pseudo-RGB representation. A compact MicroNet-SE classifier with squeeze-and-excitation blocks is used to recalibrate the fused semantic channels and perform malware classification. Experiments on the CICMalDroid 2020 and CIC-AndMal2017 datasets show that the proposed framework achieves 99.01% accuracy, 99.15% precision, 98.99% recall, and 99.07% F1-score, with only 99.6 k parameters and a model size of 1.26 MB. After conversion to TensorFlow Lite, the MicroNet-SE classifier achieves average on-device inference latencies ranging from 1.85 ms to 2.16 ms on two real mobile devices. The model also maintains stable performance under synthetic feature perturbations and practical APK-level obfuscation settings. These findings suggest that combining multi-view static semantics with denoising-based representation learning can improve both detection robustness and deployment efficiency. Overall, the results indicate that the proposed framework provides an effective and lightweight static screening component for Android malware detection in resource-constrained mobile and edge environments.

Keywords: deep learning; autoencoders; Android malware detection; feature extraction; convolutional neural networks



Academic Editor: Leandros Maglaras

Received: 23 May 2026

Revised: 24 June 2026

Accepted: 28 June 2026

Published: 1 July 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Android has become a dominant platform for mobile communication, digital payments, social networking, online education, industrial applications, and IoT-related services. Its openness, flexible application distribution model, and continuous network connectivity have greatly expanded its functionality, but they also increase the attack surface exposed to malicious applications. Android applications commonly request access to sensitive resources such as contacts, location, storage, device identifiers, network interfaces, and background services. Once these permissions or connectivity channels are abused, users

may suffer privacy leakage, financial loss, unauthorized data transmission, service disruption, or device performance degradation. Therefore, accurate and efficient Android malware detection remains an important security problem for protecting both individual users and mobile service ecosystems.

These concerns are consistent with prior studies showing that Android malware commonly abuses permissions, network connectivity, sensitive data access, and background services to conduct privacy theft, financial fraud, and other harmful activities [1,2].

These risks are not limited to technical compromise. For ordinary users, malicious applications may silently collect personal information, trigger unauthorized network communication, subscribe to paid services, display intrusive advertisements, or degrade device performance and battery life. For organizations and service providers, compromised Android devices may become entry points for credential theft, data leakage, and disruption of connected mobile services. These user-level and service-level consequences motivate malware detection methods that are not only accurate but also lightweight enough for practical deployment.

In early research, static and dynamic analysis methods based on machine learning faced challenges such as code obfuscation techniques, as well as high resource consumption and long processing times during dynamic loading. In recent years, riding the wave of advancements in visual image processing, many researchers have begun to apply visual analysis methods to Android malware detection. This approach examines the behavioral characteristics of malware from a macro level; it does not require the separate extraction of sensitive information from application code but can effectively capture the features of malicious activities simply by analyzing the texture of images. For example, the DexRay method [3] converts DEX files into one-dimensional grayscale images for classification training, achieving an accuracy rate of 97%; Sun et al. [4] converted executable Android APK files into one-dimensional fused RGB images and used CNNs for classification training, achieving an accuracy of 95%. Zhang et al. [5] combined DEX data with bytecode images from AndroidManifest.xml and used TCNs for classification training, improving accuracy to 95.44%. Ding et al. [6] utilized the bytecode images from DEX files and employed a CNN with higher-order feature maps for classification, achieving an accuracy of 95.1%.

However, most existing image mapping-based detection methods rely mainly on a single DEX representation or local byte-level texture, resulting in limited semantic coverage and sensitivity to injected noise. Lightweight Android malware detectors also often optimize the classifier alone while leaving the front-end static feature representation vulnerable to redundant or obfuscated code. To address these limitations, this paper proposes a lightweight malware detection framework that jointly integrates multi-view static feature extraction, SDAE-based feature purification, and channel-aware MicroNet-SE classification. The main contribution of the framework lies in the efficient integration of existing techniques into a task-specific multi-view static feature fusion pipeline. API calls, permission requests, and system components are represented as complementary semantic views; each view is denoised and compressed by an independent SDAE; and the purified latent representations are aligned as the R, G, and B channels of a pseudo-RGB tensor. Unlike the method by Wang et al. [7], which performs multi-feature image fusion mainly through grayscale texture enhancement, and that by Ma et al. [8], which focuses on lightweight classification with a ShuffleNetV2-ECA topology, our method combines view-level denoising and attention-based channel recalibration in a single lightweight pipeline.

Within the Android malware detection literature, this study focuses on lightweight static screening for malicious applications, a direction closely related to prior work on Android malware characterization, static analysis, and malware evolution [1,2,9]. Its practical value lies in improving the early identification of security risks that may lead to privacy

leakage, unauthorized network activity, financial loss, and device-level disruption. Previous static-analysis methods, such as Drebin, demonstrated the value of broad manifest- and code-level features for Android malware detection, while recent image mapping and lightweight deep learning approaches have explored visual representations and compact classifiers. However, existing methods often emphasize either feature fusion or classifier compression separately. The contribution of this study is to connect these two directions by introducing a view-specific denoising and fusion pipeline in which API calls, permissions, and system components are purified independently and then recalibrated through a lightweight channel-aware classifier.

The main contributions of this paper can be summarized as follows:

- We propose a lightweight Android malware detection framework that integrates multi-view static semantics with SDAE-guided feature purification. Unlike single-view image mapping methods, the framework represents API calls, permission requests, and system components as complementary security-related views, denoises each view independently, and maps the purified representations into pseudo-RGB channels with explicit semantic correspondence.
- We design a compact MicroNet-SE classifier tailored to the $48 \times 48 \times 3$ fused tensor. By combining shallow convolutional blocks, SE-based channel recalibration, Global Average Pooling, and a small output head, the classifier reduces the parameter size to approximately 0.1 M while preserving high recall.
- We provide an empirical evaluation showing that the proposed framework achieves competitive detection performance, low classifier complexity, and improved robustness under both synthetic feature perturbations and practical APK-level obfuscation. These results contribute to the development of deployable static malware screening methods for resource-constrained Android security scenarios.

The remainder of this article is organized as follows: Section 2 reviews related work in Android malware detection, encompassing static, dynamic, and hybrid analysis methods. Section 3 details the proposed detection framework, specifically covering feature extraction, feature reconstruction via SDAE, and MicroNet-SE. Section 4 presents the experimental evaluation, including dataset descriptions, performance metrics, and a comparative analysis of the results. Finally, Section 5 summarizes the paper and discusses limitations as well as future research directions.

2. Related Work

This section reviews related work from three perspectives: static analysis, dynamic analysis, and hybrid analysis. This organization reflects the main technical routes used in Android malware detection and helps position the proposed framework within lightweight static security screening.

2.1. Static Analysis

The theoretical basis of static Android malware detection is that malicious behavior leaves observable traces in application structure before execution. These traces may appear in requested permissions, declared components, intent filters, API calls, opcode sequences, embedded strings, or control-flow patterns. Prior studies and surveys have shown that such static artifacts are useful indicators for characterizing Android malware and understanding its evolution [1,9]. Static analysis therefore treats an APK as a structured artifact whose manifest-level declarations and code-level features can reveal security-relevant intent. Compared with dynamic analysis, static analysis is usually more scalable and easier to deploy because it does not require executing applications in a sandbox. However, its

effectiveness depends on whether the extracted features are sufficiently representative and robust against obfuscation, packing, reflection, and dynamically loaded code [2,9].

In practice, static analysis identifies potential malicious behavior by parsing the code, resource files, configuration files, and other information within an APK through decompilation, without actually running the application. Niall McLaughlin [10] proposed an Android malware detection system based on deep convolutional neural networks (CNNs). By performing static analysis on raw opcode sequences obtained from a disassembler to classify malware, the system achieved an accuracy rate of 98%. Wei Wang [11] proposed a hybrid model based on deep autoencoders (DAE) and convolutional neural networks (CNN). The DAE was used to reconstruct high-dimensional features of Android applications (APPs), and multiple CNNs were employed to detect Android malware. Compared to traditional machine learning methods, this approach significantly improved both the accuracy and efficiency of Android malware detection. Xiaofei Xing [12] combined the grayscale image representation of malware with autoencoder networks in deep learning models. He analyzed the feasibility of a method for analyzing grayscale malware images based on autoencoder reconstruction error and utilized the dimension reduction properties of autoencoders to achieve malware classification, with an accuracy rate of 96%. Brahami Menaouer [13] proposed a deep learning model for mobile malware detection and classification. It employs a data dimensionality reduction algorithm based on SAE. Subsequently, a CNN is utilized to detect and classify malicious applications on Android devices; experiments demonstrate that the classification performance achieves a high accuracy of approximately 98.50%. Tianyue Zhang [14] proposed an intrusion detection method (SIGMOD) based on stacked sparse autoencoders and an improved Gaussian mixture model. This method employs the Pearson correlation coefficient for linear dimensionality reduction, followed by a stacked sparse autoencoder for nonlinear dimensionality reduction to obtain low-dimensional features. Finally, it determines whether a sample is anomalous based on the energy threshold of the output from an improved Gaussian mixture model. Experimental results on the UNSW-NB15 dataset achieved an accuracy of 96.70%. Zhiqiang Wang [7] proposed an end-to-end Android malware detection technique based on RGB images and multi-feature fusion. This method extracts DEX files, androidManifest.xml files, and API calls from APK files, converts them into grayscale images, and then uses techniques such as Canny edge detection to enhance their texture features and combine them into RGB images containing multi-feature fusion information. Finally, a classification model is used to classify them, achieving an accuracy rate as high as 97.25%, outperforming existing detection methods that rely solely on DEX files as classification features. Runze Ma et al. [8] proposed a lightweight Android malware detection framework, MCADS, which employs an enhanced MLP for preliminary classification combined with a two-layer ShuffleNetV2-ECA architecture for precise classification. It extracts static features from AndroidManifest.xml and DEX files, respectively, ultimately achieving a detection accuracy of 98.12% and meeting the requirements for lightweight deployment on mobile devices.

2.2. Dynamic Analysis

Dynamic analysis complements static analysis by observing how an application behaves during execution. This is particularly useful for malware that hides malicious logic through packing, reflection, dynamic code loading, or condition-triggered behavior. Instead of relying only on pre-execution artifacts, dynamic methods monitor runtime signals such as system calls, network traffic, file operations, inter-process communication, CPU and memory usage, and user-interface interactions. These behavioral traces can reveal malicious activities that are difficult to identify from static code alone, such as data exfiltration, command-and-control communication, privilege abuse, or delayed malicious payload

execution. In practice, dynamic analysis runs an application in a controlled virtual environment or on a real device and determines whether it has malicious intent by monitoring its runtime behavior. Min Zheng et al. [15] proposed DroidTrace, a ptrace-based Android dynamic analysis system that utilizes forward execution techniques and system call monitoring, specifically designed to trigger and detect complex dynamic loading behaviors in malware. Andrea Saracino et al. [16] designed a multi-layered behavioral detection system called MADAM, which achieves efficient identification and interception of malicious Android applications in real-world environments by correlating runtime characteristics across four layers: kernel, application, user, and packet. Rajan Thangaveloo et al. [17] proposed the DATDroid dynamic analysis framework, which extracts key behavioral features—such as system calls, CPU and memory usage, and network packets—during application runtime and combines them with machine learning algorithms to achieve high-accuracy malware classification. Roopak Surendran et al. [18] proposed the GSDroid feature representation mechanism. By converting the sequence of system calls during application runtime into a directed dependency graph and utilizing graph signal processing techniques to extract low-dimensional features, they effectively addressed the high-dimensional sparsity problem and improved the detection efficiency of the classifier. Hashida et al. [19] developed the DynaMalDroid detection framework, which dynamically captures system calls using the Monkey and strace tools. By combining chi-square feature engineering with various machine learning classifiers, they achieved highly accurate detection of Android malware. Arunab Singh et al. [20] developed an interpretable dynamic Android malware detection framework that uses Linux kernel system call frequencies as features. They employed RF and XGBoost models combined with class weights and undersampling to address data imbalance, and utilized LIME technology to reveal core contributing features. This approach is particularly suitable for highly imbalanced scenarios where the proportion of malware is extremely low. K. V. Krishnam Raju [21] proposed a dynamic Android malware detection method centered on network behavior. Based on an imbalanced dataset containing 355,630 data points, the study compared 10 machine learning models, with the gradient boosting model ultimately performing best, achieving an accuracy of 97.42%. Although dynamic analysis provides richer behavioral evidence than static analysis, it usually requires sandbox execution, trigger coverage, and higher computational cost. These limitations make purely dynamic approaches less suitable for lightweight first-stage screening, but they remain valuable for validating or complementing static detection results.

2.3. Hybrid Analysis

Hybrid detection is a comprehensive defense strategy that combines static and dynamic analysis. Since malware developers often use code packing to bypass static scans or employ anti-VM techniques to evade dynamic sandboxes, hybrid detection breaks this cycle through multi-dimensional feature fusion. Zhenlong Yuan [22] proposed a method that integrates static and dynamic detection, building the DroidDetector malware detection engine based on a Deep Belief Network (DBN), achieving a detection accuracy of 96.76%, which significantly outperforms traditional machine learning models and mainstream antivirus software. Ahmed R. Nasser et al. [23] proposed the DL-AMDet Android malware detection architecture, which integrates static and dynamic analysis. It first performs preliminary detection using a CNN-BiLSTM model that extracts permission and API call features. For applications classified as benign, it then performs in-depth detection using a deep autoencoder anomaly detection model that extracts system call features, achieving an accuracy of 99.935% on datasets such as CICMalDroid 2020. However, due to the time-consuming nature of dynamic analysis and the model's heavy computational load, it is currently only suitable for off-device detection and is difficult to deploy.

3. Methods

3.1. Feature Extraction

In the process of detecting Android malware, traditional static feature extraction methods suffer from two significant issues: first, the extracted features are not comprehensive enough to accurately characterize the software's true attributes; second, these features are easily influenced by artificially added noise, leading to detection failures. To address these issues, we have adopted a targeted feature extraction method that combines features from multiple perspectives during the feature extraction phase.

Referring to the standard Android APK package structure shown in Figure 1, we first used Androguard 3.3.5 as the automated APK unpacking and static parsing tool to process the APK outer wrapper and extract the bytecode file 'classes.dex', which contains the application's core runtime logic, and the 'AndroidManifest.xml' file, which records the application's global configuration information. To comprehensively capture the software's static characteristics, our extraction tool employs regular expression matching and abstract syntax tree traversal techniques to precisely extract key features. Specifically, from the AndroidManifest.xml file, we parse all permissions requested by the application, as well as the interrelationships among the four major system components (activities, services, broadcast receivers, and content providers). Using this information, we can determine the application's resource usage intent and system architecture characteristics. At the same time, we disassemble the 'classes.dex' file to obtain Smali bytecode, from which we extract the complete call chains for all calls to sensitive Android system interfaces (APIs), thereby clarifying the application's actual runtime logic. Due to the wide variety and inconsistent formats of the extracted features, as well as issues with data fragmentation and inconsistency, data redundancy and processing difficulties are likely to arise. To address this, feature occurrence frequencies are counted during preprocessing to construct fixed vocabularies for the API, permission, and component views. During model training, the supervised Chi-square feature selector is fitted only on the training partition with a pruning ratio of 0.5 and then applied unchanged to the test partition.

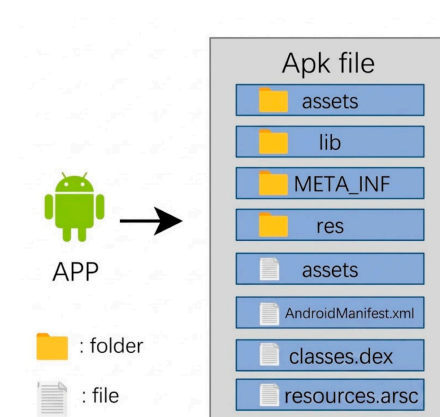


Figure 1. Android APK package structure.

3.2. Autoencoder

An autoencoder (AE) [24] is an unsupervised neural network model, as shown in Figure 2.

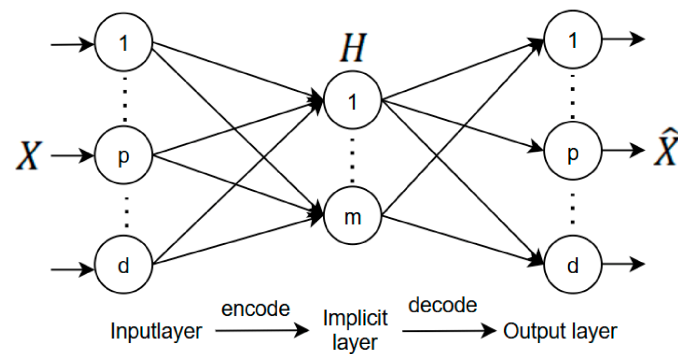


Figure 2. Basic Structure of Autoencoders.

Its primary objective is to extract the intrinsic manifold structure of high-dimensional data through feature dimensionality reduction and representation learning. A classic AE consists of an encoder and a decoder: The encoder compresses the input data X into a low-dimensional latent space via a nonlinear mapping, generating latent variables H . For a given input vector X , W_1 and W_2 represent the weight matrices of the encoder and decoder, respectively; b_1 and b_2 are the corresponding bias vectors; and $f(\cdot)$ and $g(\cdot)$ denote nonlinear activation functions. The nonlinear mapping process of the encoder can then be expressed as:

$$H = f(W_1 X + b_1) \quad (1)$$

The decoder reconstructs the latent variable H into the output $\hat{X}$, a process that can be expressed as:

$$\hat{X} = g(W_2 H + b_2) \quad (2)$$

The model updates its parameters by minimizing the reconstruction error between the input X and the reconstructed output $\hat{X}$ (such as the mean squared error, MSE), thereby forcing the latent variable H to preserve the core statistical distribution of the original data.

However, traditional AE tends to fall into the “identity mapping” trap when processing high-dimensional sparse data, meaning that the network merely memorizes the input features without extracting deep semantic information capable of generalization. To overcome this limitation, Denoising Autoencoders (DAE) [25] artificially inject random disturbances (such as Gaussian noise) into the input data during the training phase to construct corrupted inputs. Specifically, the original input $\hat{X}$ is corrupted using a specific noise distribution q , and the process can be expressed as:

$$\hat{X} \sim q(\hat{X}X) \quad (3)$$

Subsequently, the network is constrained to reconstruct the original clean signal X from the corrupted signal $\hat{X}$. At this point, the optimization objective of the DAE shifts to minimizing the error between the noise-free input and the network’s reconstructed output, and the denoising loss function is defined as:

$$L_{DAE} = \frac{1}{N} \sum_{i=1}^N \|X^{(i)} - g(f(\hat{X}^{(i)}))\|^2 \quad (4)$$

This mechanism requires the model not only to encode the local features of the input data but also to capture the global dependencies among these features in order to infer missing information. By connecting the hidden layers of multiple DAEs in a hierarchical manner, a Stacked Denoising Autoencoder (SDAE) is formed; its deep architecture can

abstract the input signal layer by layer, effectively capturing the higher-order nonlinear features of complex data.

In the proposed framework, the SDAE is used as a feature purification module rather than as a general dimensionality-reduction tool. After feature selection, each static feature view is represented as a sparse binary vector. For each view, Gaussian noise is injected during SDAE training, and the model is optimized to reconstruct the corresponding clean vector. The encoder maps the input into a 1024-dimensional dense latent representation, while the decoder reconstructs the original feature space. After unsupervised training, the decoder is discarded and only the encoder output is retained.

For pseudo-RGB construction, the 1024-dimensional latent vector of each view is reshaped into a 32×32 matrix and then upsampled to 48×48 through bicubic interpolation. The API, permission, and component views are stacked as the R, G, and B channels, respectively, forming a $48 \times 48 \times 3$ tensor. Figure 3 summarizes this transformation from sparse 1D feature vectors to a unified pseudo-RGB representation.

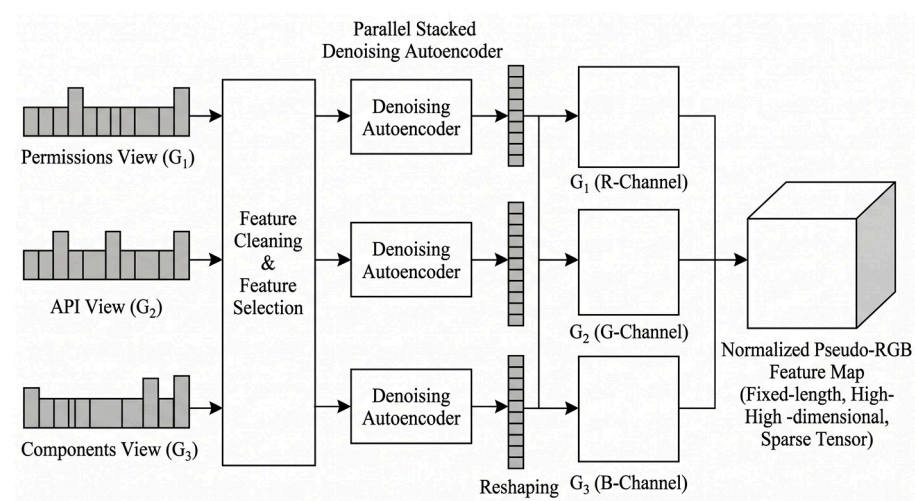


Figure 3. Generation process of the pseudo-RGB feature map.

In this study, SDAE is not claimed as a new neural component; rather, its contribution lies in view-specific feature purification before visual fusion. API calls, permissions, and system components are processed by independent SDAE branches, so that each semantic view is denoised and compressed before being aligned into the pseudo-RGB representation.

3.3. MicroNet-SE Architecture

The MicroNet-SE classifier is designed specifically for the $48 \times 48 \times 3$ pseudo-RGB representation generated by the SDAE stage. Unlike generic visual backbones designed for natural images, the proposed classifier adopts a shallow convolutional topology to reduce parameter redundancy and better match the compact malware feature maps. The network consists of three convolutional blocks. Each block contains a 3×3 convolution, Batch Normalization, ReLU activation, and a Squeeze-and-Excitation (SE) block. MaxPooling is applied after the first two blocks, reducing the feature resolution from 48×48 to 24×24 and then to 12×12 , while the channel depth increases from 32 to 64 and finally to 128.

Because the three input channels correspond to different semantic views, namely API calls, permissions, and system components, the SE-Block is introduced to perform adaptive channel recalibration. It first applies Global Average Pooling to squeeze each feature map into a channel descriptor, and then estimates nonlinear channel weights through a two-layer bottleneck structure:

$$s = \sigma(W_2 \delta(W_1 z)) \quad (5)$$

Here, δ represents the ReLU activation function, σ represents the sigmoid activation function, and W_1 and W_2 are the weight matrices for dimension reduction and dimension expansion, respectively. Next, these weight vectors are used to recalibrate (reweight) the channel dimensions of the original feature map U :

$$\hat{X}_c = s_c \cdot U_c \quad (6)$$

In the equation, $\hat{X}_c$ represents the output feature map of the c th channel after recalibration, and s_c is the corresponding attention weight scalar. This mechanism compels the network to adaptively evaluate the discriminative power of different feature views during training, actively suppressing failed channels contaminated by confounding codes or noise, and dynamically amplifying key channels containing explicit malicious logic. Thus, it achieves feature purification and recalibration with virtually no increase in computational complexity.

Figure 4 illustrates the internal structure of the customized SE-Block. The block summarizes spatial information through a squeeze operation, estimates channel-wise importance weights through the excitation module, and rescales the original feature maps accordingly. This mechanism helps MicroNet-SE emphasize discriminative semantic views while suppressing less informative or noisy channels.

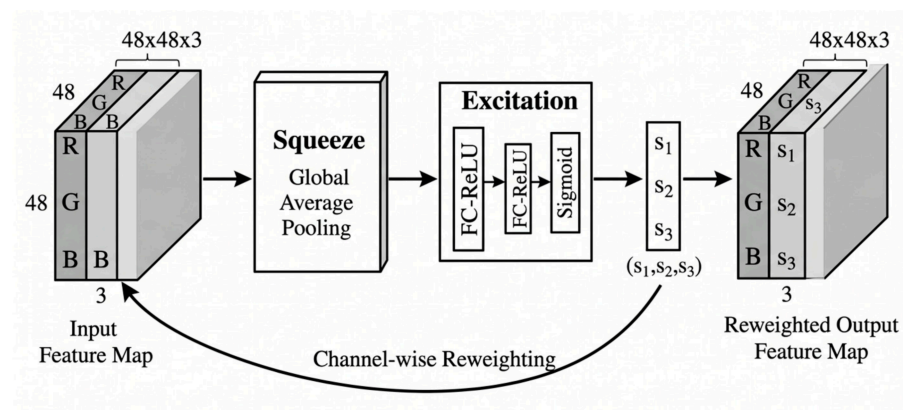


Figure 4. Internal architecture of the custom SE-Block.

Figure 5 presents a simplified overview of the proposed MicroNet-SE architecture and its main tensor transformations. For clarity, each convolutional block is shown as a single module containing convolution, batch normalization, ReLU activation, and an SE operation. The first two blocks are followed by 2×2 max-pooling operations, reducing the feature resolution from 48×48 to 24×24 and then to 12×12 , while the channel depth increases from 32 to 64 and finally to 128. After the third block, the classifier head applies Global Average Pooling (GAP), Dropout, and a Dense layer with Sigmoid activation for binary classification. The GAP operation for the c -th feature map can be expressed as:

$$y_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W X_c(i, j) \quad (7)$$

where y_c denotes the scalar output obtained after pooling the c -th channel, and $X_c(i, j)$ represents the activation value of the feature map at the spatial coordinates (i, j) . In the specific topological configuration of MicroNet-SE, the spatial parameters are fixed as $H = 12$ and $W = 12$, while the channel index spans $C \in [1, 128]$. Through this spatial integration, GAP converts the final feature tensor into a 128-dimensional vector for classification. Subsequently, a Dropout layer with a rate of 0.4 is applied to introduce stochastic

regularization, before a final Dense layer with a Sigmoid activation function outputs the ultimate binary probability of the application being malicious or benign. This structural modification reduces the number of fully connected parameters, compresses the classifier to approximately 0.1 million parameters, and improves its suitability for lightweight inference.

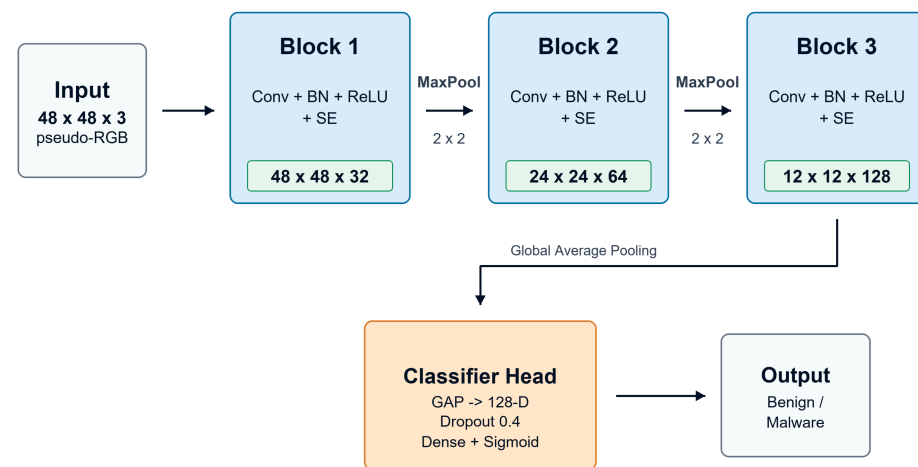


Figure 5. Simplified architecture and main tensor transformations of the proposed MicroNet-SE.

3.4. Proposed Framework

Figure 6 presents the complete workflow of the proposed Android malware detection framework from raw APK input to the final benign/malicious prediction. The framework contains four main stages: targeted static feature extraction, Boolean feature encoding, SDAE-based feature purification and pseudo-RGB mapping, and MicroNet-SE-based classification. In the first stage, AndroidManifest.xml and classes.dex are parsed to extract permission, component, and API call views. These views are then encoded as fixed-length binary vectors and processed by parallel SDAE encoders. The resulting latent representations are mapped into the R, G, and B channels of a $48 \times 48 \times 3$ pseudo-RGB tensor, which is finally classified by MicroNet-SE with SE-based channel recalibration. The following paragraphs describe these stages in more detail.

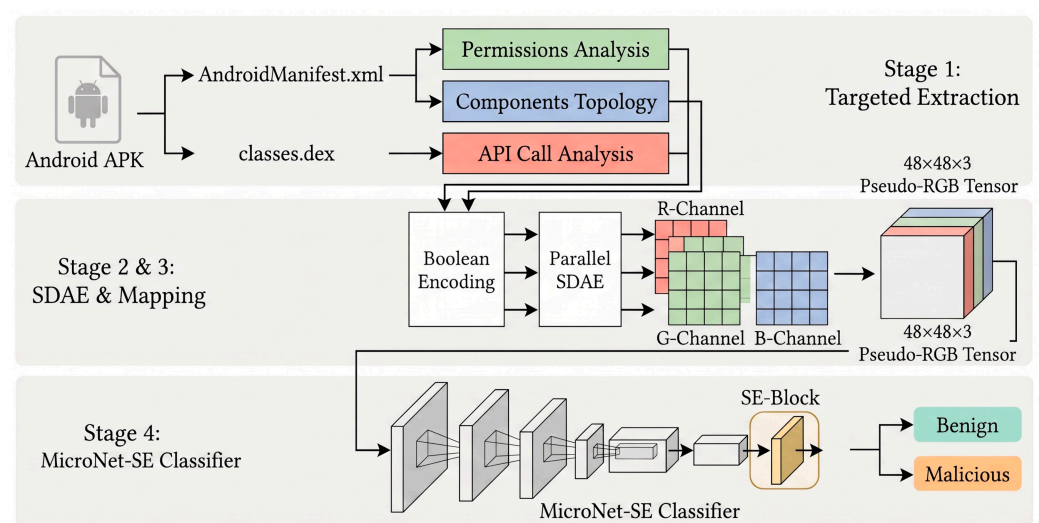


Figure 6. Overall Framework Flowchart.

In the first stage, the static analysis module extracts three types of high-dimensional sparse feature vectors—API calls, permission requests, and system components—from

the original APK file. Because these raw static features may be affected by obfuscation and redundant code, they are processed by three parallel stacked denoising autoencoders (SDAEs). During feature learning, Gaussian noise is added to the raw vectors to simulate feature contamination. Each SDAE compresses the corrupted sparse vector into a 1024-dimensional dense latent representation and reconstructs the clean signal from the corrupted input. This process helps suppress unstable or redundant features and preserve more robust behavioral representations. Next, the reconstructed features are spatially rearranged and fused across multiple views. The reconstructed feature matrices are then upsampled to a 48×48 spatial resolution via bicubic interpolation and stacked depth-wise to synthesize a $48 \times 48 \times 3$ pseudo-RGB representation. In this representation, the API, permission, and component views are mapped to separate channels, which preserves their semantic differences while providing a unified input format for 2D convolution.

Figure 7 visualizes representative SDAE-reconstructed pseudo-RGB feature maps from benign and malicious test samples. The samples are displayed in enlarged form and grouped by class label. This qualitative visualization illustrates that the three-channel representation preserves different activation patterns across sample types. Benign samples generally show more balanced channel responses, whereas malicious samples tend to contain more localized high-intensity activations related to sensitive API or component usage.

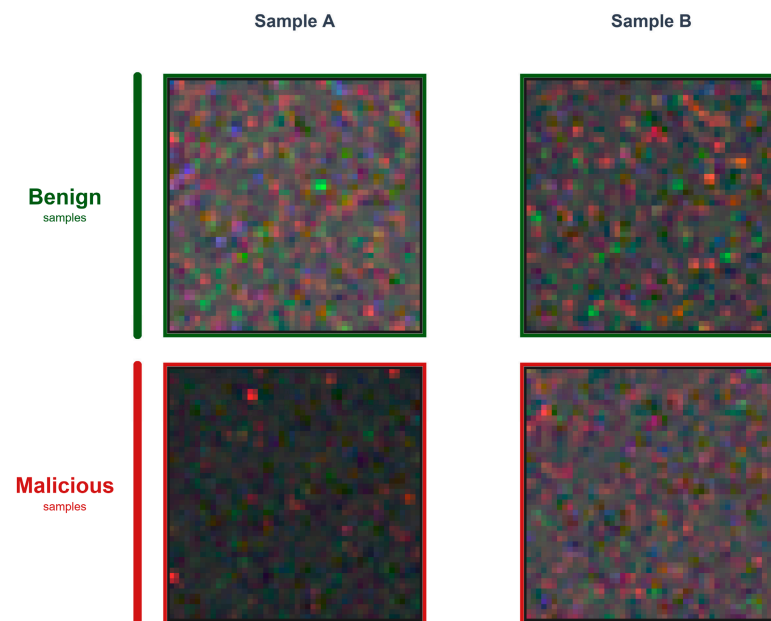


Figure 7. Enlarged visualization of SDAE-reconstructed pseudo-RGB feature maps for benign and malicious samples.

The synthesized pseudo-RGB feature maps are then fed into MicroNet-SE for classification. Since each channel corresponds to a distinct semantic view, the embedded SE-Blocks adaptively reweight API, permission, and component information according to their discriminative contribution. This design combines front-end SDAE-based feature purification with back-end lightweight channel recalibration.

4. Experiment

4.1. Dataset and Experimental Setup

All experimental data used in this study are sourced from public datasets released by the Canadian Institute for Cyber Security, specifically CICMalDroid 2020 [26] and CIC-AndMal2017 [27]. The raw APK files from the two datasets were merged and organized into benign and malicious categories. The combined dataset comprises 5835 benign and

6464 malicious Android application samples. Both datasets are public Android malware benchmarks and have been used in previous malware detection studies. Their public availability improves the reproducibility of the experimental setting and allows the results to be compared with related work. To improve data reliability, APK samples that could not be parsed successfully during static feature extraction were excluded from model evaluation, and the same feature extraction procedure was applied consistently to all retained APKs. For each APK, `AndroidManifest.xml` and `classes.dex` were parsed to extract three static feature views: requested permissions, system components, and invoked API method names. These features were converted into three aligned sparse binary matrices corresponding to the permission, component, and API views. The dataset was randomly shuffled and partitioned into training and testing sets using an 8:2 stratified split with a fixed random seed of 42. In the training pipeline, the supervised Chi-square feature selector was fitted only on the training partition and then applied unchanged to the test partition. This prevents supervised feature-selection leakage from the test set. Because malware family identifiers were not retained in the processed dataset, this study uses a sample-level stratified split rather than a malware-family-wise split. We acknowledge that possible family overlap between training and testing partitions may lead to optimistic performance estimates, and family-wise evaluation will be considered in future work. In addition, although CICMalDroid 2020 and CIC-AndMal2017 are widely used public benchmarks, their collection periods are relatively old compared with the current Android malware landscape. Therefore, the reported results should be interpreted as benchmark-based evaluation rather than definitive evidence of performance on the newest Android malware families.

The model training pipeline consists of two primary phases. In the unsupervised SDAE phase, the 2500-dimensional sparse binary vectors were corrupted with a Gaussian noise factor of 0.2 to simulate adversarial environments. Three parallel Stacked Denoising Autoencoders (SDAEs) were optimized via Mean Squared Error (MSE) loss using the Adam optimizer to compress the corrupted inputs into a 1024-dimensional dense latent space. Each SDAE was trained for 15 epochs with a batch size of 64. The refined latent vectors were then reshaped and upsampled to a 48×48 spatial resolution via bicubic interpolation, and the three views were stacked depth-wise to synthesize the normalized pseudo-RGB tensor. In the supervised classification phase, the proposed MicroNet-SE architecture was trained from scratch for 60 epochs with a batch size of 64. We employed Binary Cross-Entropy with a 0.1 label smoothing factor as the objective loss function and optimized the weights using the Adam optimizer with an initial learning rate of 0.001. To address potential overfitting and class imbalance, a Dropout rate of 0.4 and dynamically computed balanced class weights were applied. Furthermore, a ReduceLROnPlateau scheduler (decay factor = 0.2, patience = 5 epochs) and an EarlyStopping mechanism (patience = 12 epochs) were used to promote stable convergence. To ensure a consistent comparison, all baseline models were evaluated under the same hardware environment, data partitioning protocol, and optimization framework. MobileNet-V2 and EfficientNet-B4 were initialized with ImageNet pre-trained weights, whereas ShuffleNet-V2 and the proposed MicroNet-SE were trained from scratch. The reported inference latency refers to the classifier prediction time on preprocessed pseudo-RGB tensors, rather than the complete end-to-end APK analysis time. In practical deployment, the preprocessing stage introduces additional computational overhead, including APK unpacking, `AndroidManifest.xml` parsing, DEX bytecode parsing, static feature extraction, feature dictionary matching, feature selection, and SDAE encoding. These operations are more time-consuming than a single forward pass of the lightweight classifier, especially for large APKs or applications with multiple DEX files. Therefore, the proposed framework is best interpreted as a lightweight classification architecture built

on top of a static feature extraction pipeline. For real-world deployment, preprocessing can be performed offline, on an edge server, or as a batch scanning stage, while the trained MicroNet-SE classifier provides fast detection once the pseudo-RGB representation has been generated. All reported results are based on the fixed split described above. The current version of the experiment was not repeated with multiple random seeds, so standard deviations and confidence intervals are not reported. Future work will repeat the complete training and evaluation pipeline under multiple random seeds to provide more rigorous statistical validation.

4.2. Evaluation Metrics

To validate the effectiveness of the detection model proposed in this paper, the detection results are presented using a confusion matrix. The confusion matrix provides the number of true positives, true negatives, false positives, and false negatives, as shown in Table 1. True positives (TP) refer to the number of malicious programs correctly identified as such; false positives (FP) refer to the number of benign applications incorrectly identified as malicious; false negatives (FN) represent the number of malicious software samples classified as benign; true negatives (TN) represent the number of benign applications correctly identified as such; Based on the entries of the confusion matrix, four primary classification metrics—Accuracy, Precision, Recall, and F1-score—are employed to evaluate the detection performance of the proposed model and the compared methods.

Table 1. Confusion matrix.

	Predictions	
	Benign	Malicious
Benign	TN	FP
Malicious	FN	TP

Accuracy is the proportion of correctly predicted samples out of the total number of samples. The formula is as follows:

$$accuracy = \frac{TP + TN}{TP + FP + FN + TN} \quad (8)$$

Precision is the ratio of correctly predicted positive samples to the total number of predicted positive samples. It is calculated as follows:

$$precision = \frac{TP}{TP + FP} \quad (9)$$

The recall rate is the ratio of correctly predicted positive samples to the total number of positive samples, and is calculated as follows:

$$recall = \frac{TP}{TP + FN} \quad (10)$$

The F1 score is the weighted harmonic mean of precision and recall, calculated as follows:

$$F1\text{-score} = 2 \times \frac{precision \times recall}{precision + recall} \quad (11)$$

4.3. Ablation Experiment

This ablation study aims to quantitatively assess the individual contribution of each core functional module to the overall detection performance, covering feature mapping modalities, denoising mechanisms, and attention components. We evaluated six distinct model variants by systematically isolating specific functional modules: Grayscale + Baseline CNN, Pseudo-RGB + Baseline CNN, SDAE + Pseudo-RGB + Baseline CNN, SDAE + MicroNet (w/o SE), SAE + MicroNet-SE, and the full SDAE + MicroNet-SE model. This controlled-variable approach enables us to disentangle and quantify the separate impacts of SDAE-based prior denoising, multi-view Pseudo-RGB visual mapping, and the channel-calibrated topology of MicroNet-SE. We measured comprehensive quantitative metrics including accuracy, precision, recall, and F1-score, with all results compiled in Table 2.

Table 2. Detection results from different models.

Method	Accuracy	Precision	Recall	F1-Score
Grayscale + CNN	98.56%	98.91%	98.38%	98.64%
RGB + CNN	98.65%	98.99%	98.53%	98.76%
SDAE + CNN	98.85%	98.77%	99.07%	98.92%
SDAE +w/o SE-Block	95.52%	99.83%	91.72%	95.61%
SAE + MicroNet-SE	98.40%	98.98%	97.99%	98.48%
SDAE + MicroNet-SE	99.01%	99.15%	98.99%	99.07%

Figure 8 further shows that most benign and malicious samples are correctly classified, with only a small number of misclassified cases. The ablation results show that changing from grayscale mapping to multi-channel RGB mapping brings only a modest improvement, whereas introducing SDAE-based representation learning leads to a clearer performance gain. The comparison between SAE + MicroNet-SE and SDAE + MicroNet-SE suggests that the denoising constraint helps the encoder learn more stable representations from sparse Boolean features, instead of simply reconstructing the input. Removing the SE-Block reduces recall to 91.72%, indicating that adaptive weighting of API, permission, and component channels is useful for malware detection. Finally, the MicroNet-SE design uses Global Average Pooling to reduce the number of classifier parameters while maintaining high precision and recall. These results support the combined use of SDAE-based feature purification and SE-based channel recalibration in the proposed framework.

4.4. Robustness Testing

Robustness is particularly important for Android malware detection because attackers may deliberately modify an APK to reduce the visibility of malicious features while preserving its harmful functionality. Such evasion does not only introduce random noise; it may alter identifier names, string constants, bytecode layout, manifest declarations, API invocation patterns, and the accessibility of static features. Therefore, a detector that performs well on clean test samples may still fail when the same malicious logic is repackaged or obfuscated. To provide a more informative robustness evaluation, this section considers two complementary settings: synthetic feature perturbation and practical APK-level obfuscation.

First, Gaussian noise with different intensities was added to the test feature vectors to simulate imperfect static feature extraction and low-level feature contamination. The purpose of this experiment is to compare the tolerance of different feature representations under the same synthetic perturbation setting. The experimental results are shown in Table 3.

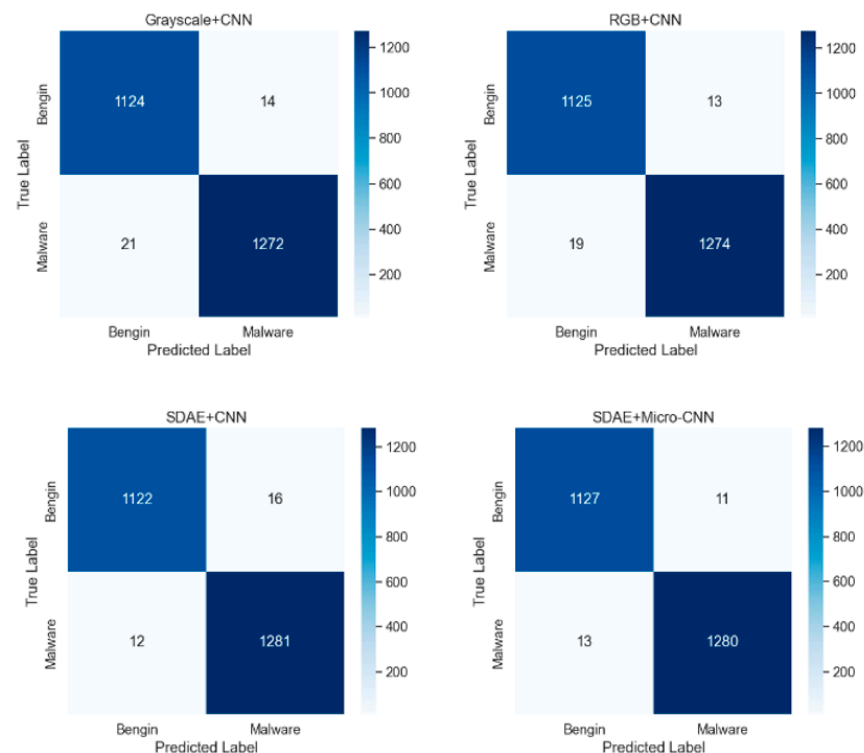


Figure 8. Confusion matrix for the model.

Table 3. Robustness Results under Synthetic Feature Perturbations.

Noise Level	Grayscale	RGB	SDAE + RGB
0.0	97.82%	93.42%	98.85%
0.1	94.67%	86.92%	95.73%
0.2	81.37%	67.54%	85.89%
0.3	52.28%	54.63%	84.37%
0.4	48.99%	53.27%	78.65%
0.5	48.25%	53.19%	64.79%

As shown in Figure 9, the proposed SDAE + RGB method maintains higher accuracy than the grayscale and raw RGB baselines as the synthetic perturbation level increases. Under 30% Gaussian noise, the proposed method still achieves 84.37% accuracy, and under 50% noise it maintains 64.79% accuracy. These results suggest that SDAE-based denoising and SE-Block-based channel recalibration improve tolerance to synthetic feature contamination. More specifically, the SDAE stage does not merely reduce dimensionality; it also encourages the latent representation to preserve stable feature patterns when part of the sparse input is corrupted. However, Gaussian perturbation should be interpreted as a controlled stress test rather than a complete simulation of practical Android malware obfuscation. To further examine robustness under more realistic transformations, we conducted an additional APK-level obfuscation experiment involving identifier renaming, string encryption, and mixed obfuscation, as shown in Table 4.

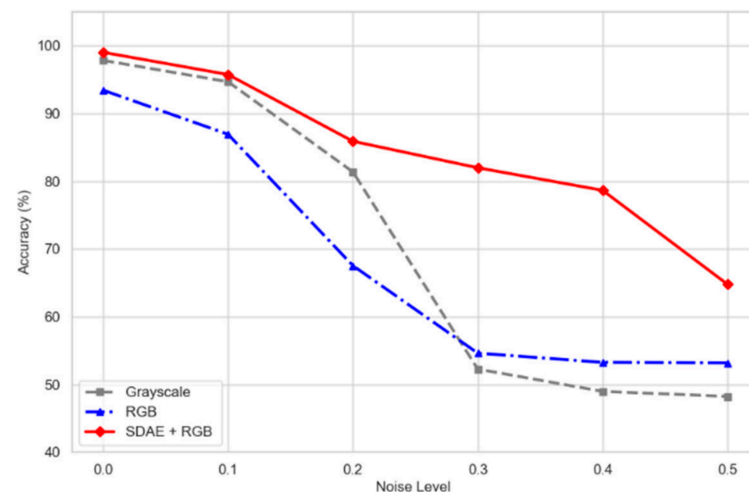


Figure 9. Robustness under Synthetic Gaussian Feature Perturbations.

Table 4. Robustness Results under Practical APK-Level Obfuscation.

Scenario	Selected APKs	Evaluated APKs	Detected Malware	Recall	FNR	Mean Malware Probability
Original	200	194	194	100.00%	0.00%	0.999980
Identifier Renaming	200	81	80	98.77%	1.23%	0.982561
String Encryption	200	127	127	100.00%	0.00%	0.999985
Mixed Obfuscation	200	129	128	99.22%	0.78%	0.988837

As shown in Table 4, the proposed framework maintains high malware recall under practical APK-level obfuscation. It achieves 98.77% recall under identifier renaming, 100.00% under string encryption, and 99.22% under mixed obfuscation. The false negative rates remain low, ranging from 0.00% to 1.23%. These results suggest that the framework does not rely only on fragile lexical identifiers or isolated string patterns. Instead, the combination of API call, permission, and component views allows the model to preserve discriminative evidence even when part of the APK representation is transformed. The SDAE stage further reduces the influence of unstable or redundant features, while the SE-based classifier can recalibrate the contribution of different semantic channels during prediction.

The practical obfuscation experiment also has several limitations. Not all APKs could be successfully rebuilt after black box obfuscation, which is a common limitation of APK rewriting tools. Therefore, the reported obfuscation results are based on successfully obfuscated and parseable APKs, while rebuild failures were recorded separately and excluded from the recall calculation. In addition, the current experiment focuses on identifier renaming, string encryption, and their mixed setting. More advanced evasion techniques, such as packing, reflection-based API hiding, dynamic code loading, and native-code protection, require further investigation.

4.5. Comparison Study

To evaluate the proposed MicroNet-SE model, we compare it with three lightweight CNN baselines: MobileNet-V2, EfficientNet-B4, and ShuffleNet-V2. For MobileNet-V2 and EfficientNet-B4, ImageNet pre-trained weights were adopted to stabilize optimization, while all models were evaluated on the same train–test split and under the same hardware conditions. The evaluation metrics comprehensively cover classification performance (accuracy, precision, recall, F1-score) and computational overhead (number of model

parameters, memory footprint, and per-sample inference latency). Specific comparison data are shown in Table 5.

Table 5. A Comparative Study of Four Lightweight CNN Models.

Model	Accuracy	Precision	Recall	F1-Score	Avg. Time	Size (MB)	Parameters
MobileNet-V2	98.35%	98.91%	97.99%	98.45%	0.61 ms	26.26	2259.3 k
EfficientNet-B4	98.60%	99.14%	98.22%	98.68%	3.20 ms	203.04	17,675.6 k
ShuffleNet-V2	98.52%	99.45%	97.76%	98.60%	0.13 ms	1.78	140.5 k
MicroNet-SE	99.01%	99.15%	98.99%	99.07%	0.36 ms	1.26	99.6 k

For classification performance, MicroNet-SE achieved 99.01% accuracy and 98.99% recall, ranking highest among the evaluated lightweight CNN models in Table 5. EfficientNet-B4 and MobileNet-V2 also achieved strong results after transfer learning, but their larger parameter counts and deeper downsampling paths are less well matched to compact malware feature maps. The results suggest that a shallow topology combined with SE-based channel calibration can capture discriminative API, permission, and component patterns while keeping the classifier small.

In terms of classifier-level computational overhead, traditional heavyweight networks are less suitable for resource-constrained scenarios. EfficientNet-B4 has 17,675.6 k parameters and requires 203.04 MB of storage in our implementation, while MobileNet-V2 contains 2259.3 k parameters and requires 26.26 MB. Although ShuffleNet-V2 achieves the lowest classifier inference latency, its recall is lower than that of MicroNet-SE. The proposed MicroNet-SE model achieves a better balance between performance and resource consumption: with only 99.6 k parameters (approximately 1/22 of MobileNet-V2) and a model size of 1.26 MB, it maintains sub-millisecond classifier inference latency of 0.36 ms while outperforming the compared baseline models in classification accuracy. This latency denotes the per-sample prediction time on preprocessed pseudo-RGB tensors and does not include APK parsing or static feature extraction. Therefore, the result reflects the lightweight nature of the classifier, while the end-to-end deployment cost also depends on the preprocessing pipeline.

To further support the lightweight deployment claim, we evaluated the MicroNet-SE classifier on two real mobile devices: Huawei Mate30 5G and Samsung Galaxy A51 5G. The trained model was converted to TensorFlow Lite in float32 format and tested using preprocessed $48 \times 48 \times 3$ pseudo-RGB tensors. Each device was evaluated with 50 warm-up runs and 1000 measured inference runs. The results are reported in Table 6.

Table 6. Classifier-level inference latency on workstation and real mobile devices.

Platform	Configuration	Runs	Avg. Latency	Std. dev.	P95 Latency	Model Size
Workstation	Intel i7-14650HX / RTX 4060	1000	0.36 ms	-	-	1.26 MB
Huawei Mate30	Kirin 990 5G, 8 GB RAM	1000	1.8455 ms	0.0452 ms	1.9277 ms	1.26 MB
Samsung Galaxy A51 5G	Exynos 980 8 GB RAM	1000	2.1553 ms	0.0323 ms	2.2090 ms	1.26 MB

As shown in Table 6, the MicroNet-SE classifier achieves average inference latencies of 1.8455 ms on the Huawei Mate30 5G and 2.1553 ms on the Samsung Galaxy A51 5G. The corresponding P95 latencies are 1.9277 ms and 2.2090 ms, respectively. Although these mobile latencies are higher than the workstation latency, the classifier still runs within a low

millisecond range on real mobile devices. These results provide additional evidence that the classifier component is suitable for lightweight mobile deployment. The mobile device experiment measures only classifier inference on preprocessed pseudo-RGB tensors and does not include APK parsing, static feature extraction, feature selection, or SDAE encoding.

Figure 10 shows the validation accuracy curves of the four lightweight CNN models during training. MicroNet-SE converges steadily and achieves the best final accuracy under this experimental setting. This result suggests that, for the proposed SDAE-reconstructed pseudo-RGB representation, a compact architecture tailored to small feature maps can be more effective than directly adopting larger natural-image backbones.

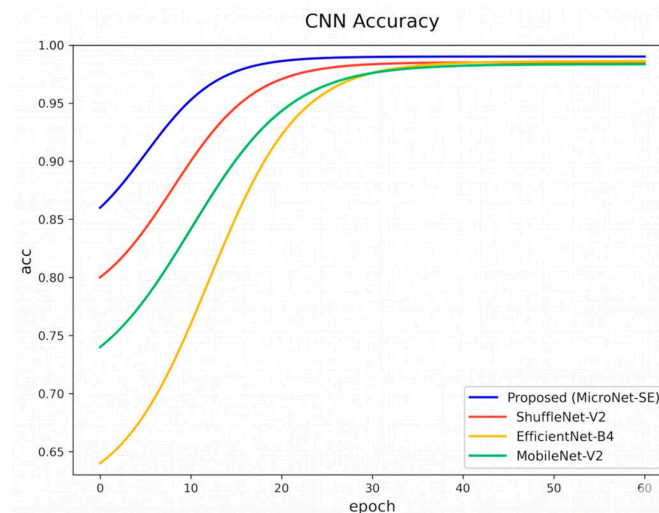


Figure 10. Accuracy of different lightweight CNN models.

In addition to the lightweight CNN baselines evaluated under the same experimental protocol, we compared the proposed framework with representative Android malware detection studies, including Sun et al. (2021) [4], Alazab et al. [28], Yadav et al. (2022) [29], Drebin (Arp et al., 2014) [30], Ma et al. [8], Alfaw et al. (2025) [31], and the other methods listed in Table 7. Because these studies differ in dataset composition, preprocessing, implementation details, hardware platforms, and timing protocols, the comparison is used as contextual reference rather than as a strictly identical benchmark.

Table 7. Comparison results with the existing works.

Model	Platform	Accuracy	Precision	Recall	F1-Score	Avg. Time	Size (MB)	Parameters
Drebin [30]	Workstation	90.24%	86.76%	97.43%	91.79%	6.41 s	1.45	179 k
Sun et al. [4]	Workstation	90.78%	91.47%	93.94%	92.69%	2.12 s	12.31	1016.90 k
Alazab et al. [28]	Workstation	93.14%	91.16%	98.52%	92.48%	10.18 s	0.13	-
Yadav et al. [29]	Workstation	91.28%	90.64%	97.34%	90.20%	25.98 s	66.96	17,552.20 k
Ma et al. [8]	Workstation	98.12%	99.04%	97.71%	98.37%	12.76 s	30.34	7976.17 k
Alfaw et al. [31]	Android TFLite	98.50%	-	-	-	45 ms	<50	-
Our	Android TFLite	99.01%	99.15%	98.99%	99.07%	1.85–2.16 ms	1.26	99.6 k

As shown in Table 7, the proposed framework achieves competitive detection performance, with an accuracy of 99.01%, precision of 99.15%, recall of 98.99%, and F1-score of 99.07%. Alfaw et al. (2025) [31] is included as a recent deployment-oriented comparison because it reports Android TensorFlow Lite results for mobile malware detection. In their deployment experiment, the CNN model achieved 98.50% accuracy with 45 ms inference time on mid-range devices. By comparison, the proposed MicroNet-SE classifier achieves an average on-device inference latency of 1.85–2.16 ms after TensorFlow Lite conversion.

The comparison in Table 7 is intended to provide deployment-related context rather than a strictly controlled benchmark. Most earlier studies report workstation-level or paper-reported results, whereas Alfaw et al. (2025) [31] and the proposed method include Android TensorFlow Lite deployment measurements. For Alfaw et al., the mobile deployment experiment reports accuracy, inference time, and memory usage, while the remaining metrics are not provided and are marked as “-”. The reported “<50 MB” value refers to Android application memory usage rather than standalone model size. These differences should be considered when interpreting the results.

5. Conclusions

This study investigated whether lightweight Android malware detection can be improved by treating static APK evidence as complementary semantic views rather than as a single byte-level or texture-like representation. The proposed framework combines API calls, permission requests, and system components through SDAE-guided feature purification, pseudo-RGB mapping, and MicroNet-SE classification. The main interpretation of the results is that robust lightweight detection depends not only on reducing classifier size, but also on constructing a stable and discriminative front-end representation before classification.

The experimental results support this interpretation. The proposed framework achieves 99.01% accuracy, 99.15% precision, 98.99% recall, and 99.07% F1-score with only 99.6 k parameters and a model size of 1.26 MB. These results indicate that the multi-view static representation and lightweight classifier can maintain high detection performance while reducing model complexity. The ablation study further shows that SDAE-based denoising and SE-based channel recalibration improve performance by preserving stable multi-view feature patterns and emphasizing informative semantic channels. The robustness experiments under synthetic perturbation and APK-level obfuscation also show that the framework retains high malware recall when part of the input representation is corrupted or transformed. Taken together, the classification results, complexity analysis, and robustness evaluation support the conclusion that the proposed framework is suitable as a lightweight static screening component.

From a deployment perspective, the proposed framework is best viewed as a lightweight static screening component for Android malware detection in mobile, IoT, or edge-assisted security scenarios. Once APK features have been extracted and transformed into pseudo-RGB tensors, the MicroNet-SE classifier provides fast malware prediction with low storage overhead. The complete end-to-end cost, however, still depends on preprocessing operations such as APK parsing, manifest analysis, DEX bytecode parsing, feature extraction, and SDAE encoding. For high-risk cases, the proposed method should therefore complement deeper dynamic or hybrid analysis rather than replace it. Future work will further measure complete end-to-end latency, memory consumption, and energy cost on real Android devices, including APK parsing and feature extraction.

Several limitations remain. First, the current experiments use a sample-level stratified split rather than a malware-family-wise split, which may lead to optimistic generalization estimates if related malware variants appear in both training and testing partitions. Second, although practical obfuscation experiments were added, the evaluation is limited to successfully rebuilt and parseable APKs. More diverse obfuscation, packing, reflection, and dynamic loading techniques should be further investigated. Third, the current framework relies primarily on static features. Future work will validate the framework using malware-family-wise splitting and newer temporally separated Android malware datasets, and will explore hybrid static–dynamic detection by integrating lightweight runtime behavior features, such as system calls, network behavior, or dynamic loading traces.

Author Contributions: Conceptualization, M.H.; methodology, M.H. and Y.L.; software, Y.S.; validation, Y.S. and M.H.; writing—original draft preparation, Y.S.; and writing—review and editing, M.H., Y.S. and Y.L. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Key Research and Development Project of the Sichuan Provincial Department of Science and Technology (2023YFG0171), the Fundamental Research Funds for the Central Universities (24CAFUC01009), and the China Higher Education Industry-University-Research Innovation Fund: New Generation Information Technology Innovation Project (2024IT079).

Data Availability Statement: The Android malware detection dataset is publicly available on the Canadian Cybersecurity Institute at the following link: <https://www.unb.ca/cic/datasets/index.html> (accessed on 28 December 2025).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Zhou, Y.; Jiang, X. Dissecting Android Malware: Characterization and Evolution. In *Proceedings of the IEEE Symposium on Security and Privacy*; IEEE: New York, NY, USA, 2012; pp. 95–109.
2. Suarez-Tangil, G.; Tapiador, J.E.; Peris-Lopez, P.; Ribagorda, A. Evolution, Detection and Analysis of Malware for Smart Devices. *IEEE Commun. Surv. Tutor.* **2014**, *16*, 961–987.
3. Daoudi, N.; Samhi, J.; Kabore, A.K.; Allix, K.; Bissyandé, T.F.; Klein, J. Dexray: A simple, yet effective deep learning approach to android malware detection based on image representation of bytecode. In *Proceedings of the International Workshop on Deployable Machine Learning for Security Defense*; Springer: Cham, Switzerland, 2021; pp. 81–106.
4. Sun, T.; Daoudi, N.; Allix, K.; Bissyandé, T.F. Android malware detection: Looking beyond dalvik bytecode. In *Proceedings of the 2021 36th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW)*; IEEE: New York, NY, USA, 2021; pp. 34–39.
5. Zhang, W.; Luktarhan, N.; Ding, C.; Lu, B. Android malware detection using tcn with bytecode image. *Symmetry* **2021**, *13*, 1107. [\[CrossRef\]](#)
6. Ding, Y.; Zhang, X.; Hu, J.; Xu, W. Android malware detection method based on bytecode image. *J. Ambient. Intell. Humaniz. Comput.* **2023**, *14*, 6401–6410.
7. Yu, Q.; Wang, Z.; Ju, L.; Yuan, S.; Zhang, Y. Android Malware Detection Technology Based on SC-ViT and Multi-Feature Fusion. In *Proceedings of the 2024 IEEE 23rd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*; IEEE: New York, NY, USA, 2024; pp. 1105–1114.
8. Ma, R.; Yin, S.; Feng, X.; Zhu, H.; Sheng, V.S. A lightweight deep learning-based android malware detection framework. *Expert Syst. Appl.* **2024**, *255*, 124633. [\[CrossRef\]](#)
9. Tam, K.; Khan, S.J.; Fattori, A.; Cavallaro, L. The Evolution of Android Malware and Android Analysis Techniques. *ACM Comput. Surv.* **2017**, *49*, 76. [\[CrossRef\]](#)
10. McLaughlin, N.; Martinez del Rincon, J.; Kang, B.; Yerima, S.; Miller, P.; Sezer, S.; Safaei, Y.; Trickel, E.; Zhao, Z.; Doupé, A. Deep android malware detection. In *Proceedings of the Seventh ACM on Conference on Data and Application Security and Privacy*; ACM: New York, NY, USA, 2017; pp. 301–308.
11. Wang, W.; Zhao, M.; Wang, J. Effective android malware detection with a hybrid model based on deep autoencoder and convolutional neural network. *J. Ambient. Intell. Humaniz. Comput.* **2019**, *10*, 3035–3043.
12. Xing, X.; Jin, X.; Elahi, H.; Jiang, H.; Wang, G. A malware detection approach using autoencoder in deep learning. *IEEE Access* **2022**, *10*, 25696–25706. [\[CrossRef\]](#)
13. Menaouer, B.; Islem, A.E.H.M.; Nada, M. Android malware detection approach using stacked autoencoder and convolutional neural networks. *Int. J. Intell. Inf. Technol. (IJIT)* **2023**, *19*, 1–22. [\[CrossRef\]](#)
14. Zhang, T.; Chen, W.; Liu, Y.; Wu, L. An intrusion detection method based on stacked sparse autoencoder and improved gaussian mixture model. *Comput. Secur.* **2023**, *128*, 103144. [\[CrossRef\]](#)
15. Zheng, M.; Sun, M.; Lui, J.C. DroidTrace: A ptrace based Android dynamic analysis system with forward execution capability. In *Proceedings of the 2014 International Wireless Communications and Mobile Computing Conference (IWCMC)*; IEEE: New York, NY, USA, 2014; pp. 128–133.
16. Saracino, A.; Sgandurra, D.; Dini, G.; Martinelli, F. Madam: Effective and efficient behavior-based android malware detection and prevention. *IEEE Trans. Dependable Secur. Comput.* **2016**, *15*, 83–97. [\[CrossRef\]](#)
17. Thangaveloo, R.; Jinga, W.W.; Lenga, C.K.; Abdullaha, J. Datdroid: Dynamic analysis technique in android malware detection. *Int. J. Adv. Sci. Eng. Inf. Technol.* **2020**, *10*, 536–541. [\[CrossRef\]](#)

18. Surendran, R.; Thomas, T.; Emmanuel, S. Gsdroid: Graph signal based compact feature representation for android malware detection. *Expert Syst. Appl.* **2020**, *159*, 113581. [[CrossRef](#)]
19. Manzil, H.H.R. Dynamaldroid: Dynamic analysis-based detection framework for android malware using machine learning techniques. In *Proceedings of the 2022 International Conference on Knowledge Engineering and Communication Systems (ICKES)*; IEEE: New York, NY, USA, 2022; pp. 1–6.
20. Singh, A.; Tanha, M.; Girdhar, Y.; Hunter, A. Interpretable Android Malware Detection Based on Dynamic Analysis. In *Proceedings of the ICISSP*; Science and Technology Publications: Setúbal, Portugal, 2024; pp. 195–202.
21. Raju, K.K.; Akula, D.; Areti, S.S.; Barre, B.K.; Bhupathiraju, S.S.K.R. Android Malware Detection Using Dynamic Analysis. In *Algorithms in Advanced Artificial Intelligence*; CRC Press: Boca Raton, FL, USA, 2025; pp. 693–700.
22. Yuan, Z.; Lu, Y.; Xue, Y. Droiddetector: Android malware characterization and detection using deep learning. *Tsinghua Sci. Technol.* **2016**, *21*, 114–123. [[CrossRef](#)]
23. Nasser, A.R.; Hasan, A.M.; Humaidi, A.J. DL-AMDet: Deep learning-based malware detector for android. *Intell. Syst. Appl.* **2024**, *21*, 200318.
24. Chen, S.; Guo, W. Auto-encoders in deep learning—A review with new perspectives. *Mathematics* **2023**, *11*, 1777.
25. Manjunatha, B.; Shastry, K.A.; Naresh, E.; Pareek, P.K.; Reddy, K.T. A network intrusion detection framework on sparse deep denoising auto-encoder for dimensionality reduction. *Soft Comput.* **2024**, *28*, 4503–4517.
26. MahdaviFar, S.; Kadir, A.F.A.; Fatemi, R.; Alhadidi, D.; Ghorbani, A.A. Dynamic android malware category classification using semi-supervised deep learning. In *Proceedings of the 2020 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDDCom/CyberSciTech)*; IEEE: New York, NY, USA, 2020; pp. 515–522.
27. Khan, Z.I.; Afzal, M.M.; Shamsi, K.N. A comprehensive study on CIC-IDS2017 dataset for intrusion detection systems. *Int. Res. J. Adv. Eng. Hub* **2024**, *2*, 254–260. [[CrossRef](#)]
28. Alazab, M.; Alazab, M.; Shalaginov, A.; Mesleh, A.; Awajan, A. Intelligent mobile malware detection using permission requests and API calls. *Future Gener. Comput. Syst.* **2020**, *107*, 509–521.
29. Yadav, P.; Menon, N.; Ravi, V.; Vishvanathan, S.; Pham, T.D. EfficientNet convolutional neural networks-based Android malware detection. *Comput. Secur.* **2022**, *115*, 102622.
30. Arp, D.; Spreitzenbarth, M.; Hubner, M.; Gascon, H.; Rieck, K.; Siemens, C. Drebin: Effective and explainable detection of android malware in your pocket. In *Proceedings of the Ndss*; Internet Society: Geneva, Switzerland, 2014; pp. 23–26.
31. Alfaw, A.; Rouached, M.; Akremi, A. A Resilient Deep Learning Framework for Mobile Malware Detection: From Architecture to Deployment. *Future Internet* **2025**, *17*, 532. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.